# Работа с URLSession

Поход в сеть

Сергей Дунаев | Архитектор

# Сергей Дунаев

Архитектор, Социальная платформа

## О себе

- Закончил МЭИ, Автоматика и вычислительная техника
- ВШЭ, Фондовый рынок и инвестиции
  В iOS с 2012 года
- Основатель онлайн-школы SwiftLab
- Хоби: дайвинг, горные лыжи, фортепиано

# Ключевые моменты

- HTTP и REST API
- Знакомство с URLSession
- Codable и парсинг JSON
- Обработка ошибок
- Архитектура сетевого слоя
- Загрузка изображений с прогрессом

# Что такое сетевые запросы и зачем они нужны в мобильных приложениях?

## Сетевой запрос

Это обращение из приложения к внешнему серверу по определенному протоколу (чаще всего HTTP или HTTPS), с целью получить или отправить информацию.

## Примеры

- Получение списка новостей
- Загрузка изображений
- Отправка формы регистрации

## Почему это важно

- Практически любое приложение работает с сетью.
- Без сетевого взаимодействия невозможно реализовать:
  - Актуальные данные
  - Персонализацию
  - Обновления контента
  - Чаты, комментарии и т.д.

## Роль клиента и сервера

- **Клиент** — это наше iOS-приложение.
- **Сервер** — удалённая система, хранящая и обрабатывающая данные.
- Приложение инициирует запрос, сервер возвращает ответ (чаще всего — JSON с нужными данными).

# Как устроены сетевые запросы

## Что такое HTTP-протокол

**HTTP (HyperText Transfer Protocol)** — это протокол передачи данных между клиентом (наше приложение) и сервером (API).

- Это основа общения в интернете: все браузеры, приложения, сайты используют HTTP или его защищённую версию HTTPS.
- Он работает по принципу “запрос → ответ”.

# Основные HTTP-методы

| Метод | Назначение | Пример использования |
|---|---|---|
| GET | Получить данные | Получение списка товаров |
| POST | Отправить данные | Отправка формы регистрации |
| PUT | Обновить данные | Редактирования профиля |
| DELETE | Удалить данные | Удаление задачи из списка |

➡️ В 90% случаев в мобильных приложениях используют GET и POST.

# Из чего состоит HTTP-запрос

- **URL** — адрес ресурса (например: https://api.example.com/users)
- **Метод** — GET*, POST, и т.д.
- **Заголовки (Headers)** — метаинформация (например, Content-Type, Authorization)
- **Тело (Body)** — данные запроса (чаще всего в формате JSON, используется в POST, PUT)

*Как правило не работает с телом, использует параметры

# HTTP-ответ

Сервер возвращает:

- **Статус-код** (например, 200 OK, 404 Not Found)
- **Заголовки**
- **Тело ответа** (чаще всего — JSON)

# Что такое REST API

REST (Representational State Transfer) — архитектурный стиль, определяющий, как строятся API.

## Принципы REST:

- Использование HTTP-методов по назначению
- Работа с ресурсами по URL-адресам
- Без сохранения состояния между запросами (stateless)
- Формат обмена — чаще всего JSON

## Примеры REST API:

```
// Получить всех пользователей
GET  https://api.example.com/users

// Получить пользователя с id 42
GET  https://api.example.com/users/42

// Создать нового пользователя
POST https://api.example.com/users
```

# HTTP-коды и их значения
## Примеры

### Успешные операции (2xx)

200 – Успешный ответ
201 – Ресурс успешно создан (обычно при POST)
204 – Запрос успешен, но ответа нет (например при DELETE)

### Перенаправления (3xx)

301 – Постоянное перенаправление
302 – Временное перенаправление
307 – Современный аналог 302
308 – Современный аналог 301

### Ошибки клиента (4xx)

400 – Неверный запрос
401 – Пользователь не авторизован
403 – Доступ запрещен
404 – Ресурс не найден

### Ошибки сервера (5xx)

500 – Ошибка на стороне сервера
502 – Проблема на прокси-сервере
503 – Сервер временно недоступен

# Знакомство с URLSession

# Что такое URLSession

URLSession — это основной API в iOS/macOS, который позволяет отправлять HTTP-запросы, загружать данные, файлы, изображения, и обрабатывать ответы.

```
let url = URL(string: "https://jsonplaceholder.typicode.com/posts")!

let task = URLSession.shared.dataTask(with: url) { data, response, error in
    if let data = data {
        print("✅ Данные получены: \(data)")
    } else if let error = error {
        print("❌ Ошибка: \(error)")
    }
}

task.resume()
```

# Как управлять HTTP-запросом с помощью URLRequest

## Что такое URLRequest

**URLRequest** — это структура, которая позволяет точно настроить HTTP-запрос: выбрать метод (GET, POST, и т.д.), добавить заголовки и тело.

- **Объект URL** — это просто адрес.
- **URLRequest** — это конкретный запрос, со всеми параметрами.

## Когда нужен URLRequest?

Всегда, когда запрос не простой GET.

Если нужно:

- Отправить POST, PUT, DELETE
- Передать заголовки (например, токен авторизации)
- Отправить JSON в теле запроса

# Пример настройки POST-запроса с JSON

```
let url = URL(string: "https://jsonplaceholder.typicode.com/posts")!
var request = URLRequest(url: url)
request.httpMethod = "POST"
request.setValue("application/json", forHTTPHeaderField: "Content-Type")
let posts = [
   "userId": "1",
   "title": "Новость дня",
   "body": "Сегодня хорошая погода!"
]
let jsonData = try? JSONSerialization.data(withJSONObject: posts)
request.httpBody = jsonData

URLSession.shared.dataTask(with: request) { data, response, error in
   if let data = data {
      print("✅ Данные получены: \(data)")
   } else if let error = error {
      print("❌ Ошибка: \(error)")
   }
}.resume()
```

# Async/Await в URLSession

```
func sendPost() async {
    ...
    do {
        let (data, response) = try await URLSession.shared.data(for: request)
        print("✅ Ответ получен:")
        if let responseString = String(data: data, encoding: .utf8) {
            print(responseString)
        }
    } catch {
        print("❌ Ошибка: \(error)")
    }
}


/// Как вызвать sendPost()

Task {
    await sendPost()
}
```

# Codable и парсинг JSON

# Что такое JSON

JSON (JavaScript Object Notation) — это стандартный формат обмена данными между клиентом и сервером

```
{
  "userId": 1,
  "id": 1,
  "title": "Новость дня",
  "body": "Сегодня хорошая погода!"
}
```

➡️ Приходит с сервера в виде Data. Нам нужно превратить его в структуру Swift

# Что такое Codable

Codable — это протокол, который объединяет Encodable (в Swift → JSON) и Decodable (из JSON → Swift)

- Работает с JSONEncoder и JSONDecoder
- Автоматически сопоставляет поля, если названия совпадают

# Пример структуры

```
struct Post: Codable {
   let userId: Int
   let id: Int?
   let title: String
   let body: String
}
```

❗ Если названия свойств не соответствуют ключам в JSON, можно использовать **CodingKeys**

```
extension Post {
   private enum CodingKeys: String, CodingKey {
      case userId = "user_id"
      case id
      case title
      case body
   }
}
```

# Парсинг данных

Парсинг данных из JSON:

```
do {
   let posts = try JSONDecoder().decode([Post].self, from: data)
   print("✅ Успешно распарсили \(posts.count) постов")
} catch {
   print("❌ Ошибка парсинга: \(error)")
}
```

Обратное действие — Encodable:

```
let newPost = Post(userId: 1, id: 101, title: "Новость", body: "Текст новости")
let jsonData = try JSONEncoder().encode(newPost)
```

# Обработка ошибок

# Обработка ошибок

Ошибки URL / соединения

```
if let error = error {
    print("❌ Ошибка соединения: \(error.localizedDescription)")
    return
}
```

Проверка HTTP-статуса ответа

```
if let httpResponse = response as? HTTPURLResponse,
   !(200...299).contains(httpResponse.statusCode) {
    print("⚠️ Ошибка HTTP: \(httpResponse.statusCode)")
    return
}
```

# Обработка ошибок

Ошибки парсинга JSON (DecodingError)

```
do {
    let posts = try JSONDecoder().decode([Post].self, from: data)
} catch let decodingError as DecodingError {
    print("❌ Ошибка парсинга: \(decodingError)")
} catch {
    print("❌ Другая ошибка: \(error)")
}
```

# Суммарный пример обработки всех ошибок

```
if let error = error {
    print("Ошибка соединения: \(error.localizedDescription)")
    return
}
guard let httpResponse = response as? HTTPURLResponse,
      (200...299).contains(httpResponse.statusCode) else {
    print("Ошибка HTTP-кода")
    return
}
guard let data = data else {
    print("Нет данных")
    return
}
do {
    let result = try JSONDecoder().decode([Post].self, from: data)
    print("Успех: \(result.count) постов")
} catch {
    print("Ошибка парсинга: \(error)")
}
```

# Архитектура сетевого слоя

# Сетевой слой и зачем он нужен

### Повторно используемый код

Один и тот же запрос можно вызывать из разных экранов / модулей без дублирования логики

### Изолировать сеть от UI

UI (вью-контроллеры) не должны знать, как именно устроен сетевой запрос.

- Вью просто говорит: «дай мне данные» — а как именно они приходят, скрыто.
- Это упрощает читаемость, повышает переиспользуемость, и делает код легче тестировать.

### Тестировать проще

Сетевой слой можно заменить моками или заглушками.

- Мы можем подменить UserService на MockUserService, возвращающий заранее заготовленные данные.
- Не нужно стучаться в реальный интернет при запуске UI-тестов.
- Быстро и стабильно проводить юнит-тесты.

### Дженерики с Codable моделями

Мы можем написать универсальный метод запроса, который работает с любой моделью Codable.

Такой подход снижает дублирование и делает SDK-интерфейс более мощным.

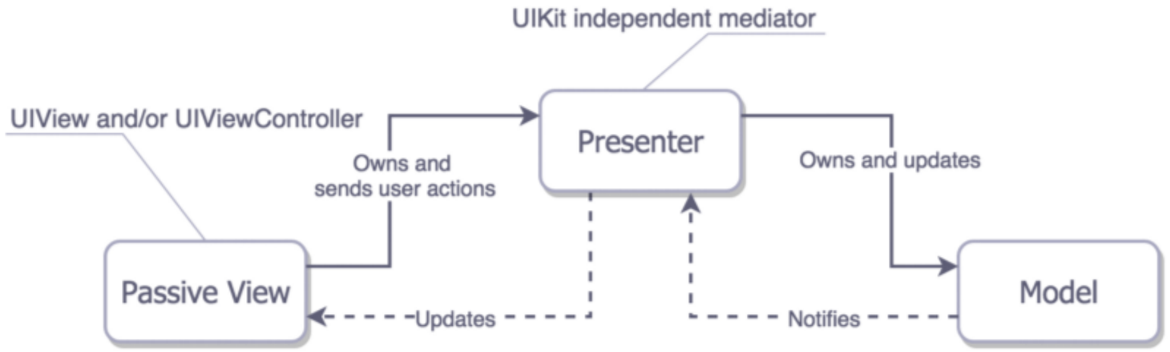
# Протокол NetworkClientInput

```
protocol NetworkClientInput: AnyObject {
    func get<ResponseSchema: Decodable>(
        urlString: String,
        completion: @escaping (Result<ResponseSchema, NetworkClientError>) -> Void
    )

    func post<ResponseSchema: Decodable>(
        urlString: String,
        requestBody: Encodable,
        completion: @escaping (Result<ResponseSchema, NetworkClientError>) -> Void
    )

    func download(
        urlString: String,
        progressBlock: ((Double) -> Void)?,
        completion: @escaping (Result<Data, NetworkClientError>) -> Void
    )
}
```

# Архитектура MVP (Model–View–Presenter)




## Краткое описание

Архитектурный шаблон, который используется для разделения логики пользовательского интерфейса (UI) от бизнес-логики в приложениях. В контексте разработки на Swift, особенно для приложений на iOS, этот шаблон помогает создать четкое разделение обязанностей между различными компонентами приложения, что упрощает тестирование и поддержку кода.

## Преимущества

- Чёткое разделение ответственности
- View становится “тупой” — только отображает
- Легче тестировать бизнес-логику (в Presenter и Model)
- Код проще читать и поддерживать

## Особенности

- Presenter не знает ничего о UIKit — только работает через интерфейсы (протоколы)
- ViewController может быть как View, так и частично View+Controller (в учебных проектах — это нормально)

# Загрузка изображений с прогрессом

# Делегаты URLSession — сводная таблица

| Протокол | Назначение | Пример использования |
|---|---|---|
| URLSessionDelegate | Базовый делегат: глобальные события, ошибки, сертификаты, завершение сессии | Проверка SSL-сертификата, обработка завершения сессии |
| URLSessionDataDelegate | Работа с данными, получение через dataTask – по частям, с прогрессом | Загрузка JSON, изображений, стриминг данных |
| URLSessionDownloadDelegate | Загрузка файлов на диск, отслеживание прогресса, сохранение временного файла | Скачивание PDF, видео, архивов, больших изображений |
| URLSessionTaskDelegate | Обработка общих событий задача: переадресация (редиректы), завершение, ошибки | Отмена задачи, логика при редиректе, контроль состояния |

Т-Банк — финансовая онлайн-экосистема, объединяющая полный спектр финансовых услуг для частных лиц и бизнеса